

Biblioteca Pillow



- Ferramenta padrão e indispensável do ecossistema Python.
- Fluxos de trabalho 100% automatizados.
- ⬡ Abertura, edição, aplicação de filtros e salvamento em escala.

Transformando horas de trabalho manual em segundos de processamento.

```
1 lista_de_fotos = ['originais/fotoA.png', 'originais/fotoB.png',  
  'originais/fotoC.png'] # lista com o nome das fotos em  
  formato .png que queremos converter para .jpg  
2  
3 for nome_da_foto in lista_de_fotos: # para cada foto dentro da  
  lista...  
4  
5     imagem_png = Image.open(nome_da_foto)  
6     imagem_png = imagem_png.convert('RGB') # transforma a  
  imagem para o modo RGB (resolve o erro 'mode P')  
7     novo_nome = nome_da_foto.replace('.png', '.jpg') # remove o  
  '.png' do nome antigo e troca por '.jpg')  
8  
9  
10    novo_nome = nome_da_foto.replace('.png', 'jpg')  
11    novo_nome = novo_nome.replace('originais/', 'modificadas/')  
  # troca a pasta originais para modificadas para salvamento  
  no local correto  
13    imagem_png.save(novo_nome) # salva no novo formato  
14    print(f'Convertido de {nome_da_foto} para {novo_nome}')  
15
```



**Conversão Batch
Automatizada**



Aplicação automatizada de filtros estéticos e anonimização de dados.

The Engine

```
1  imagem_foto1 = Image.open('originais/foto1.jpg') # abre a foto
2
3  imagem_pb = imagem_foto1.convert('L') # deixa a imagem em preto
   e branco (modo 'L' significa ganho de cinza)
4  imagem_pb.save('modificadas/resultado_pb.jpg')
5  print('Imagem em preto e branco criada.')
6
7  imagem_blur =
   imagem_foto1.filter(ImageFilter.GaussianBlur(radius=20))
   # aplica o desfoque (blur) na foto
10  imagem_blur.save('modificadas/resultado_blur.jpg')
11  print('Imagem com blur criada.')
```

I

The Gallery



Original



Modo 'L' (Grayscale)



GaussianBlur(radius=20)

Proteção de direitos autorais e geração dinâmica de elementos visuais

```
imagem_foto2 = Image.open('originais/foto2.jpg') # abre a foto

desenhar = ImageDraw.Draw(imagem_foto2) # cria uma ferramenta para desenhar em cima da imagem

try: # carrega a fonte arial no tamanho 30
    fonte = ImageFont.truetype('arial.ttf', 30)
except IOError:
    fonte = ImageFont.load_default() # caso não ache a arial, usa a padrão

desenhar.text((50, 50), 'PROPRIEDADE DA FATEC RC',
fill='white', font=fonte) # escreve o texto na posição X=50, Y=50, na cor branca

imagem_foto2.save('modificadas/resultado_marca_dagua.jpg') # salva o resultado

print('Marca d água adicionada com sucesso.')
```



Original



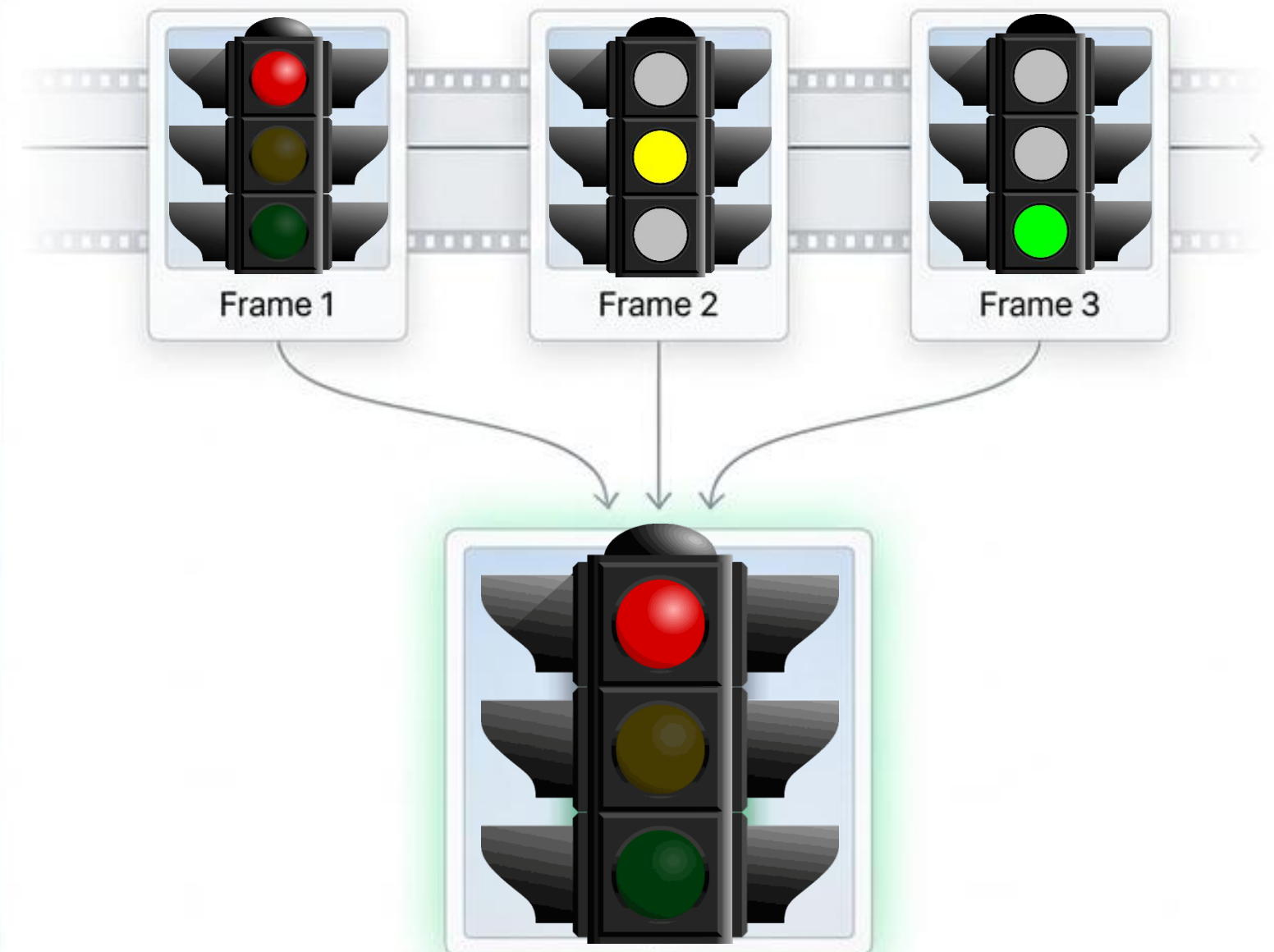
Resultado da Aplicação da Marca D'água

Orquestrando quadros estáticos para a criação de mídias dinâmicas.

The Engine

```
1  img1 = Image.open('originais/frame1.png') #
5
2  abre as 3 imagens da animação
3  img2 = Image.open('originais/frame2.png')
4  img3 = Image.open('originais/frame3.png')
5
6
7  img1.save( # salva a primeira imagem e 'engata' as
   outras duas logo atrás para criar o GIF
9      'modificadas/animacao.gif',
10     save_all=True,
11     append_images=[img2, img3],
12     duration=500, # tempo de cada foto na tela
   (500 milissegundos = 0.5 segundos)
13     loop=0      # 0 significa que o GIF vai
   ficar rodando para sempre em loop
15 )
16
17 print('GIF animado gerado com sucesso.')
```

The Gallery



animacao.gif | Loop: ∞ | 500ms/frame

Ocultação e Extração de Metadados (Atividade 5)

```
1  imagem_celular = Image.open('originais/foto_celular.heic') # abre
   a foto tirada por celular (com dados reais)
2
3  dados_ocultos = imagem_celular.getexif() # extrai os dados
   ocultos (EXIF) da imagem
4
5  print('\n--- Atividade 5: Lendo dados ocultos da imagem ---')
6
7  for codigo, valor in dados_ocultos.items(): # varre os dados,
   mostra os códigos numéricos e respectivos valores
8
9      # exemplos de códigos comuns: 271 (marca do celular), 272
   (modelo do celular)
10
11     nome_tag = ExifTags.TAGS.get(codigo, codigo) # traduz o
   código numérico para um nome legível (ex: 271 vira 'Make', 306
   vira 'DateTime')
12
13     print(f'Código do metadado: {nome_tag} -> Informação:
   {valor}')
```



--- Atividade 5: Lendo dados ocultos da imagem ---

Código do metadado: Make -> Informação: **APPLE**
Código do metadado: Model -> Informação: **IPHONE 14**
Código do metadado: Software -> Informação: **26.4.1**
Código do metadado: DateTime -> Informação: **15 DE ABRIL
DE 2026, ÀS 08H29MIN37S**
Código do metadado: GPSInfo -> Informação: **[DADOS DE
LATITUDE E LONGITUDE]**

Síntese: O Ecossistema Pillow



Leve e Rápida

Alta performance para scripts curtos e manipulação em lote.



Extremamente Robusta

Versatilidade que vai de filtros estéticos até a extração pericial de metadados.



Indispensável

A ferramenta definitiva para automação, conversão e inteligência de dados em imagens.